Seminario de C++
Parte II

Departamento de Lenguajes y Sistemas Informáticos
Universidad de Alicante
Curso 2026-2027



Esta obra se publica con una licencia BY-NC-SA Creative Commons License

Index

1. Memoria dinámica en C++

2. Operaciones canónicas con memoria dinámica

Index

1. Memoria dinámica en C++

2. Operaciones canónicas con memoria dinámica

Gestión de memoria estática

- Un objeto se crea de forma estática cuando se declara una variable de la siguiente manera:

```
Pixel p(1,2);
```

- Cuando el objeto sale de ámbito se invoca automáticamente el **destructor** del objeto.

Gestión de memoria dinámica

- Se usan las funciones **new** y **delete** y se aplican a punteros.
- Ejemplos de reserva de memoria:
  - Constructores por defecto

```
//reserva de memoria para un entero apuntado por p
int *p = new int;
//reserva de memoria para un Pixel apuntado por px
Pixel *px = new Pixel;
```

  - Otros constructores

```
int *p = new int(3);
Pixel *px = new Pixel(2,3);
```

  - Ejemplo de uso:

```
int *p = new int;
*p = 3;    //ojo, ERROR sin el new (violacion de segmento)
p[0] = 3; //equivalente
```

Liberación de memoria dinámica

- El programador **DEBE** liberar con **delete** la memoria reservada con **new**;
- **delete** se invoca sobre el puntero sobre el que se ha hecho la reserva;
- si el puntero es sobre un tipo primitivo solo queda asignarle `nullptr`;

```
int *p = new int(3);
//liberacion de memoria
if(p != nullptr){
  delete p;
  p = nullptr;
}
```

- si el puntero es sobre un objeto de una clase, se invoca automáticamente el destructor de la clase, después se asigna `nullptr` al puntero

```
Pixel *px = new Pixel;
//liberacion de memoria
if(px != nullptr){
  delete px; //se invoca al destructor de la clase Pixel
  px = nullptr;
}
```

Liberación de memoria con arrays de objetos

```
Pixel *px = new Pixel[3];

//liberacion de memoria INCORRECTA
if(px != nullptr){
  delete px; //se invoca al destructor de la clase Pixel
             //(solo para el primer objeto)
  px = nullptr;
}

//liberacion de memoria CORRECTA
if(px != nullptr){
  delete [] px; //se invoca al destructor de la clase Pixel
                //(para todos los objetos del array)
  px = nullptr;
}
```

Regla general: si la reserva de memoria se hizo con el operador **new [ ]**, la liberación se tiene que hacer con el operador **delete [ ]**.

Acceso a funciones miembro

Con memoria estática el operador de acceso a una función miembro es un punto (.)

```
Pixel p(2,3);
cout << "Coordenada X = " << p.getX() << endl;
```

Con memoria dinámica el operador de acceso a una función miembro es una flecha ($->$)

```
Pixel *px = new Pixel(2,3);
cout << "Coordenada X = " << px->getX() << endl;
                            // (*px).getX() es equivalente
```

Herramienta para depuración: valgrind

- En este caso vamos a ver qué ocurre si un programa implementado en C++ deja memoria sin liberar, una vez finalizada su ejecución.
- Para poder utilizar esta herramienta tenemos que compilar los programas con la opción de generación de información para depuración (-g).

```
valgrind [opciones] ejecutable-de-nuestro-programa
```

Lo compilamos con la opción de depuración y lo ejecutamos con valgrind:

```
g++ -g -o ejemplo ejemplo.cc
valgrind ejemplo
```

Se genera una salida que nos informa si no se ha liberado alguna memoria.

Si lo ejecutamos utilizando la opción `--leak-check=full` de valgrind, además nos indicará en que línea se reservó dicha memoria:

```
valgrind --leak-check=full ejemplo
```

Ejemplo de uso de valgrind (1)

En este caso hemos creado además una variable como Pixel *p1 = new Pixel, y no hemos invocado a delete.

```
==18465==
==18465== HEAP SUMMARY:
==18465==     in use at exit: 72,712 bytes in 2 blocks
==18465==   total heap usage: 3 allocs, 1 frees, 73,736 bytes allocated
==18465==
==18465== LEAK SUMMARY:
==18465==    definitely lost: 8 bytes in 1 blocks
==18465==    indirectly lost: 0 bytes in 0 blocks
==18465==      possibly lost: 0 bytes in 0 blocks
==18465==    still reachable: 72,704 bytes in 1 blocks
==18465==         suppressed: 0 bytes in 0 blocks
==18465== Rerun with --leak-check=full to see details of leaked memory
==18465==
==18465== For counts of detected and suppressed errors, rerun with: -v
==18465== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Ejemplo de uso de valgrind (2)

Repitiendo la ejecución anterior de `valgrind` con la opción `--leak-check=full` la salida sería:

```
==19515== HEAP SUMMARY:
==19515==     in use at exit: 72,712 bytes in 2 blocks
==19515==   total heap usage: 3 allocs, 1 frees, 73,736 bytes allocated
==19515==
==19515== 8 bytes in 1 blocks are definitely lost in loss record 1 of 2
==19515==    at 0x4C2E0EF: operator new(unsigned long) (in /usr/lib/valgrind/vgp
reload_memcheck-amd64-linux.so)
==19515==    by 0x400A70: main (mainPixel.cc:9)
==19515==
==19515== LEAK SUMMARY:
==19515==    definitely lost: 8 bytes in 1 blocks
==19515==    indirectly lost: 0 bytes in 0 blocks
==19515==      possibly lost: 0 bytes in 0 blocks
==19515==    still reachable: 72,704 bytes in 1 blocks
==19515==         suppressed: 0 bytes in 0 blocks
==19515== Reachable blocks (those to which a pointer was found) are not shown.
==19515== To see them, rerun with: --leak-check=full --show-leak-kinds=all
==19515==
==19515== For counts of detected and suppressed errors, rerun with: -v
==19515== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Ejemplo de uso de valgrind (3)

Repitiendo la ejecución anterior de `valgrind`, haciendo el delete del puntero a Pixel en el programa principal, la salida sería:

```
==19621== HEAP SUMMARY:
==19621==     in use at exit: 72,704 bytes in 1 blocks
==19621==   total heap usage: 3 allocs, 2 frees, 73,736 bytes allocated
==19621==
==19621== LEAK SUMMARY:
==19621==    definitely lost: 0 bytes in 0 blocks
==19621==    indirectly lost: 0 bytes in 0 blocks
==19621==      possibly lost: 0 bytes in 0 blocks
==19621==    still reachable: 72,704 bytes in 1 blocks
==19621==         suppressed: 0 bytes in 0 blocks
==19621== Rerun with --leak-check=full to see details of leaked memory
==19621==
==19621== For counts of detected and suppressed errors, rerun with: -v
==19621== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Index

1 Memoria dinámica en C++

2 Operaciones canónicas con memoria dinámica

Operaciones canónicas de una clase

- constructores (incluído constructor de copia)
- destructor
- operador =
- operador salida <<

```
//supongamos que tenemos esta clase sobre la que definiremos las
    operaciones canonicas
class Dato{
  private:
    int dato;
    Pixel* pix;
    Dato* sig;
};
```

Operaciones canónicas de una clase: constructores

Métodos para crear objetos. Se autoinvocan cuando se declara una variable de la clase que crea un objeto en memoria estática, o bien cuando se usa con el operador new, que crea un objeto en memoria dinámica.

```
//constructor por defecto
Dato::Dato(){
  dato = 0;
  pix = nullptr;
  sig = nullptr;
}

//constructor con parametros
Dato::Dato( int d, Pixel p ){
  dato = d;
  pix = new Pixel( p );
  sig = nullptr;
}

//declaracion de objetos de tipo Dato
Pixel p(2,3);
Dato d1(3, p); //objeto con memoria estatica
Dato* d2 = new Dato(1, p); //objeto con memoria dinamica
```

Operaciones canónicas de una clase: constructor de copia

```
//se invoca automaticamente en un paso (o devolucion) por valor
Dato::Dato( const Dato &p ){
  dato = p.dato;
  if( p.pix != nullptr ){
    pix = new Pixel( *(p.pix) ); //invocamos al constructor de
        copia de Pixel
  }
  else pix = nullptr;
  sig = nullptr;
}
//declaracion de objetos de tipo Dato
Pixel p(2,3);
Dato d1(3, p); //objeto con memoria estatica
Dato d3(d1); //d3 es una copia de d1 - se invoca al constructor
    de copia
```

Operaciones canónicas: destructor

- Se encarga de finalizar el objeto y de liberar la memoria dinámica que se hubiera reservado cuando el objeto sale de ámbito.
- Se invoca automáticamente cuando sale de ámbito un objeto o cuando se utiliza la función **delete** con un objeto.

```
Dato::~Dato(){
  dato = -1;
  if( pix != nullptr )
    delete pix;    //se invoca explicitamente al destructor
                   //de Pixel para pix
  pix = nullptr;
  sig = nullptr;
}
//uso
Pixel(2,3);
Dato d1(3, p); //objeto con memoria estatica
Dato* d2 = new Dato(1, p); //objeto con memoria dinamica
Dato d3(d1); //d3 es una copia de d1

//se invoca automaticamente a los destructores de d1 y d3
delete d2; //se invoca al destructor para d2
```

Operaciones canónicas: sobrecarga operador asignación

- Se usa para poder asignar un objeto a otro como si fueran variables de tipo primitivo, de manera que **se crea una copia del objeto**.
- El compilador genera uno que asigna atributo a atributo, pero hay que redefinirlo si la clase tiene variables de memoria dinámica.

```
Dato & Dato::operator =( const Dato &obj ) {
  if( this != &obj ){
    dato = obj.dato;
    if( pix != nullptr )
      delete pix; //IMPORTANTE liberar la memoria de pix
    if( obj.pix != nullptr ){
      pix = new Pixel( *(obj.pix) );
    }
    else pix = nullptr;
    sig = nullptr;
  }
  return *this;
}
//uso
Pixel p(2,3);
Dato d1(3, p), d2;
d2 = d1; //se invoca al operador =
         //si no estuviera definido, d1.pix y d2.pix
         //apuntarian a la MISMA zona de memoria, ERROR
```

Operaciones canónicas: sobrecarga operador salida

```
//declaracion
friend ostream & operator<<( ostream &out, const Dato &obj );

//implementacion
ostream& operator <<( ostream &out , const Dato &obj ) {
  out << obj.dato << " ->" << *(obj.pix) << endl ;
  return out;
}

//uso
Pixel p(2,3);
Dato d1(3, p);
cout << d1 << endl; // 3 -> (2,3)
```

Asignación y acceso a funciones con punteros

Un ejemplo de asignación de punteros (en lugar de objetos). En este caso los punteros de los objetos apuntarán a la misma zona de memoria (comparten toda la información).

```
//CODIGO EN UNA FUNCION MIEMBRO DE LA CLASE DATO

Dato *d1 = new Dato(2, p);
Dato *d2;

d2 = d1; // ahora d1 y d2 apuntan a la misma zona de memoria

cout<<"d2="<<(d2->pix)->getX()<<endl; // (*d2->pix).getX()
cout<<"d1="<<(d1->pix)->getX()<<endl;

d2->pix->setX(0); //modificamos el valor de la coordenada X en d2
cout<<"d2="<<(d2->pix)->getX()<<endl;
cout<<"d1="<<(d1->pix)->getX()<<endl;
```

A la salida se mostrará:

```
2
2
0
0
```

Problema en el ejemplo anterior

Se han declarado dos punteros a objetos de tipo Dato. Esto implica que, antes de salir del método, hay que liberar la memoria. Por lo tanto, y refrescando el ejemplo:

```
Dato *d1 = new Dato(2, p);
Dato *d2;

d2 = d1; // ahora d1 y d2 apuntan a la misma zona de memoria
...

delete d1;
d1 = nullptr;
d2 = nullptr;
```

¿Por qué no hacemos `delete` en d2? porque d1 y d2 apuntan a la misma zona de memoria, y la misma ha sido ya liberada a través del delete sobre d1 $\Rightarrow$ Se puede producir por tanto una violación de segmento.

Consejo

Hay que evitar este tipo de situaciones, es decir, tener dos punteros que apunten a la misma zona de memoria, salvo que sepamos bien lo que hacemos.

En el caso de objetos, este error se suele producir al **copiar** un objeto que contiene punteros y no se realiza una copia correcta (reservando una zona de memoria y copiando el contenido).

Por tanto, suele ocurrir si no están correctamente programados:

- el constructor de copia y/o
- el operador asignación